

Sparse Prefill Prediction: Research Plan and Task List

Status: active, single source of truth. **Last updated:** 4 October 2026. **Supersedes:** every earlier plan, including all versions of the "Estimator Frontier" plan. Their reasoning and referee responses are kept, unedited, in docs/archive/ for traceability; nothing in them overrides this file.

0. How to use this document

Who reads it. The researcher, Claude chat (direction and decisions) and Claude Code (implementation). All three treat it as authoritative.

Rules.

1. If anything (code, an older doc, a chat message, memory) conflicts with this plan, this plan wins until it is changed. Flag the conflict; do not silently follow the other source.
2. The plan changes only by editing docs/RESEARCH_PLAN.md and committing. The PDF is regenerated from it in the same commit (python tools/build_plan_pdf.py). Never edit the PDF by hand.
3. Task status (Section 8) is updated in the same commit as the work that changes it.
4. A new decision gets a body change **and** one dated line in Appendix A. No version numbers.
5. Direction changes (gaps, hypotheses, pairings, arms, budget) are decided in Claude chat with the researcher, then written here. Claude Code implements what is written here and asks before changing scope.

Status words. *Done:* built, tested, used. *Built:* code and tests exist, not yet used for the study. *Partly built:* some pieces exist (named). *Not started.* *Blocked* (reason given). *Decision:* needs the researcher.

1. The question

When does training-free block-sparse prefill actually pay end to end on a given host CPU and GPU, and can that be predicted before running the workload there?

Papers report sparse attention kernel speedups on one or two machines. Our own measurements show the kernel speedup often does not survive end to end: on an A100 the sparse kernel was 1.28x to 1.96x faster than FlashAttention, yet prefill became slower (0.63x to 0.75x) because the host CPU built masks on the critical path. The same configuration was faster on an L4, whose slower GPU hid the host work. Whether sparse prefill pays therefore depends on the host, the GPU and how their work overlaps, and also on whether the sparse masks keep accuracy.

Contributions, in order of weight.

- **G2 (lead).** A per-operation critical-path model that places data-dependent host work (mask building) and its GPU-to-CPU synchronisations *inside* a prefill step, built only from measured components.
- **G3 (lead).** A test of whether attention recall is an estimator-independent proxy for accuracy: at matched per-layer recall, does accuracy depend on which estimator chose the blocks?
- **G1 (validation question).** Whether this explicit model predicts end-to-end time on real, physically distinct host-GPU pairings better than a learned predictor when only a few hardware systems are available for training. Cross-device latency prediction itself is not new (CTFusion), so G1 is not claimed as a new problem.

Target. An IEEE Transactions-level journal (TC, TPDS, TCAD or ACM TACO; open, see Section 11). Nothing here claims that bar is reached.

2. What is already established (banked evidence)

These results exist and are commit-stamped. They motivate the study; they are not its confirmatory evidence.

Earlier oracle-mask study (L4, A100, H100).

Finding	Evidence and caveat
Kernel speedup does not survive end to end	A100, sparsity 0.75: kernel 1.28x (8192 tokens) and 1.96x (16384) faster than flash; end-to-end prefill with the reference mask builder 0.75x and 0.63x.
The sign depends on the card	32768 tokens, sparsity 0.75: about 1.3x on an L4, about 0.48x on an A100 with the reference builder.
The cause is host-side mask construction	L4 and A100 hosts share a CPU platform; builder costs within about 3%. A vectorised builder turned the A100 result into a speedup at 16384.
The mechanism held on a third card	Of three predictions registered before the H100 run, two passed and one failed (7 of 9 cells).
The oracle is expensive	Scoring costs about 35x the latency the sparsity saves at 32768 on an L4.
A cheap estimator trails the oracle	Mean-pool trails the oracle by 32 to 40 points at 1.5B and 39 to 76 at 7B on multi-key retrieval at 16384.

Pilots on the L4, Qwen2.5-1.5B-Instruct, 1 to 3 October 2026 (called "T4 pilots" after the audit item, not the GPU).

Finding	Evidence and caveat
Even the oracle is non-inferior only at density 0.5	Primary claim only on qa_1/16384 (70 vs 68 of 100, bound -9.0); secondary on niah_multivalue/16384 (bound -7.7). Nothing at density 0.25 or 0.1.
Inline mean-pool is non-inferior nowhere	7 to 24 points below dense at 0.5; estimator plus mask build cost 0.06 to 0.31 of the kernel saving.
Scalar-threshold XAttention is non-inferior nowhere	At $\tau = 0.95$, bounds -12.7 to -50.8 at densities 0.11 to 0.39; niah_multivalue/32768 collapsed to 0 of 50. Dense reproduced 400 of 400.
Density depends on the prompt at fixed τ	33% of blocks on qa_1 vs 11% on NIAH essays at 32768.
Estimator cost depends on the code path	On the L4's torch fallback, XAttention's estimate costs more than dense attention (19.5 vs 14.5 ms at 16384).
Calibrated XAttention	Built and pre-registered; not run (task T1.1).

Standing caveats. Some A100 dense baselines were estimated; some 32768-token rows exceed the position limit; results span three mask-rule eras that must not be pooled; the pilots cover one model, one card and synthetic tasks.

3. Gaps and nearest work

Gap	Statement	Nearest work (read in full unless marked)
G1	Learned cross-device predictors exist (CTFusion: 20 training systems, host side is input preprocessing). KernelSight-LM measures host constants per host and assumes they do not depend on the GPU, without testing it. Untested: whether an explicit critical-path model of measured components predicts data-dependent sparse prefill better than a learned predictor when few systems are available.	CTFusion, KernelSight-LM, CloserToMe; nn-Meter, HELP (abstracts only)
G2	Host-overhead summaries (TaxBreak's HDBI, SKIP's TKLQT) are ratios or sums; KernelSight-LM uses one maximum per step plus a serial tail; serving simulators (LLMServingSim 2.0, the APEX simulator) work from per-device profiles and model no host work inside a step. TaxBreak says its replay can be imperfect for synchronisation-heavy kernels. No lightweight model places data-dependent mask generation and its synchronisations inside a prefill step.	TaxBreak, SKIP/TKLQT, KernelSight-LM, LLMServingSim 2.0, APEX (Fan et al.), APEX simulator (Lin et al.)

Gap	Statement	Nearest work (read in full unless marked)
G3	Whether recall is a sufficient accuracy proxy across estimators is untested. The Sparse Frontier lists it as unknown. VSPrefill and Less Is More compare recall across methods, not accuracy at matched recall. AB-Sparse and CompactAttention compare accuracy at fixed budgets or per-method operating points. Oracle-guided sparse prefill names recall of oracle support as a diagnostic it has not yet run.	VSPrefill, The Sparse Frontier, Less Is More, oracle-guided sparse prefill, AB-Sparse, CompactAttention

The full related-work list with read status is in docs/research_brief.pdf (the professor brief). Positioning must be re-checked in IEEE Xplore and Scopus before submission (task T7.3).

4. Hypotheses

Each has a statistic and a tolerance fixed at lock (marked **[lock]**). Failures are reported as failures. Results are reported per pairing; nothing is pooled into a significance test across pairings.

#	Hypothesis	Test and falsifier
H0	Separability. Host components measured with one GPU equal those measured with another GPU on the same host platform; device components measured on one host equal those on another host for the same GPU. Link components are pairing-specific.	Host side: P1 vs P2, and P3 vs P5 vs P6. Device side: P1 vs P3 (L4), P4 vs P6 (H100). Per-operation differences within [lock] , after provider-level variance from the dense workload. If H0 fails, H2 is not tested and scope narrows to components measured on the target pairing.
H1	Composition. On every pairing, the recursion over measured components predicts untraced end-to-end prefill within tolerance, and on held-out pairings beats the sum, the KernelSight-style step form and the learned predictor. Stated in advance: on dense arms (no synchronisation inside the step) the recursion and the step form should agree; an advantage is predicted only on arms with data-dependent synchronisations.	Absolute relative error within [lock] ; sparse-vs-dense sign correct outside the resolution floor; recursion error below all three baselines on held-out pairings, reported separately for dense and synchronising arms. A large advantage on dense arms points to a baseline bug, not support for G2. If the learned predictor wins, that is reported.
H2	Transfer. Predictions stay within tolerance when components come from other pairings: host components for P2 (from P1) and P5 (from P3); device components for P3 (from P1); both for P6 (host from P3 and P5, device from P4).	Same statistic as H1. P6 is the full-transfer test: only its link microbenchmarks are measured before its prediction is committed. Conditional on H0.
H3	Forecast. On P4 (a host platform used nowhere else), the crossover context length per estimator falls inside the band predicted before P4's workload runs.	Predicted band vs measured crossover with bootstrap intervals.
H4	Recall sufficiency. At matched per-layer recall, accuracy does not depend on which estimator chose the blocks, at block sizes 32, 64 and 128.	Paired comparison at matched per-layer recall, or the estimator coefficient in a pre-registered model, against an equivalence bound [lock] . Also by depth third and by head type. Either outcome is reported.
H5	Crossover direction (secondary). (a) On a fixed host, a faster GPU moves the crossover to longer contexts for estimators with host-side steps (AMD: L4, L40S, H100). (b) The shift is smaller for an estimator whose selection runs entirely on the GPU. (c) On a fixed GPU, the host with lower measured host cost gives the shorter crossover.	Signs predicted from measured component ratios before data, with tolerance stated on the input ratio. Consequences of H1, reported separately.

5. Method

5.1 Definitions

- **Unmeasured pairing.** A host-GPU pairing on which the full workload has not yet run when the prediction is written. Allowed inputs: host components measured on that host without the workload on the GPU; device components of that GPU; link microbenchmarks on the pairing (seconds). The

prediction file and its sha256 are synced before the workload starts; the analysis refuses any prediction stamped later.

- **Host component** $c_{\text{host}}(i)$: host time to issue operation i (framework dispatch, library front end, launch floor) plus host-side compute such as CPU mask building. Measured with device work replaced by null kernels (TaxBreak's protocol) and by running host-side compute directly at target sizes.
- **Device component** $c_{\text{dev}}(i)$: GPU execution time of operation i , from isolated per-operation replay on the target GPU with the arm's recorded masks; never from a run of the composed workload.
- **Link component** $c_{\text{link}}(i)$: pairing-dependent cost of operation i : host-device copies and synchronisation round trips, from microbenchmarks on the pairing.
- **Operation sequence and synchronisation map.** The ordered operations of each arm and the positions of its GPU-to-CPU synchronisations. Software-dependent, not hardware-dependent: taken from a P1 trace and checked against each pairing's kernel list.
- **Recall.** At a layer, the share of that layer's dense attention mass, computed from its actual inputs in the arm's own forward pass, that the kept blocks capture (primary). Recall against a separate dense pass (whose scores are already cached) is a secondary measure. H4 is read on the primary one.
- **Accuracy.** Paired non-inferiority against dense (test in Section 5.7).
- **Profitable.** End-to-end speedup above 1 over the fastest correct dense kernel, at non-inferior accuracy, estimator cost included.

5.2 Hardware pairings

Pairing	Host platform	GPU	Provider	Role
P1	Intel Cascade Lake	L4	GCP G2	Anchor; source of transferred components
P2	Intel Cascade Lake	A100	GCP A2	Host-side H0; host transfer (H2)
P3	AMD EPYC 3rd gen	L4	AWS G6	Device-side H0; device transfer (H2)
P5	AMD EPYC 3rd gen	L40S	AWS G6e	Host-side H0; host transfer; GPU-speed axis (H5a)
P4	Intel Sapphire Rapids	H100	GCP A3	Held-out host platform: forecast (H3)
P6	AMD EPYC 3rd gen (7R13)	H100	AWS p5.4xlarge	Device-side H0; full transfer (H2); H5a, H5c. Run last
P7 (optional)	AMD EPYC 2nd gen	A10G	AWS G5	Extra held-out point if time allows

Run order: P1, P2, P3, P5, P4, P6, then P7, so every transferred component exists before it is used. "Same platform" is not the same SKU: CPU model string, core count, SMT state and observed frequency are recorded on every row and compared before H0 is read.

5.3 Per-pairing session protocol

1. **Environment.** One container image across providers; driver matched where possible and recorded; threads pinned to the GPU's NUMA node; CPU frequency sampled; clocks locked where permitted and recorded.
2. **Host-speed probe** (fixed single-thread loop plus null-kernel launch burst) at the start, middle and end of the session and between ground-truth blocks.
3. **Link microbenchmarks.** Bandwidth both ways, pinned and pageable; 1-byte copy; synchronisation round trip; PCIe generation and width; bandwidth under a concurrent CPU load sized like the largest mask build; launch-queue depth Q .
4. **Host components.** Operation sequence with null kernels; CPU-side compute at target sizes.
5. **Device components.** Every operation of every arm replayed in isolation (masks and shapes recorded from P1; data-dependent work replayed with each prompt's recorded mask). Graph replay of capturable arms as a cross-check.
6. **Prediction committed.** Computed from steps 3 to 5 (plus transferred components for H2), written with its sha256 and synced.

7. **Ground truth.** Untraced end-to-end prefill per arm, context length and batch, arms interleaved, with a dense control.
8. **Profiler overhead.** Traced vs untraced end-to-end time per arm.

5.4 Cost model

For one CUDA stream, operation i has host cost $h_i = c_{\text{host}}(i)$, device cost $d_i = c_{\text{dev}}(i)$ (GPU execution only) and link cost $l_i = c_{\text{link}}(i)$ (copy time; zero for a kernel). H_i is when the host has issued operation i ; D_i is when the device finishes it.

```

H_i = H_(i-1) + h_i
if Q launches are pending:  H_i = max(H_i, D_(i-Q))          (finite launch queue, Q measured per
pairing)
at a synchronisation point: H_i = max(H_i, D_(i-1)) + c_link_sync
D_i = max(H_i, D_(i-1)) + l_i + d_i                          (+ measured contention, only where a
trace shows a concurrent copy)
end-to-end prefill time = D_N

```

Only device costs transfer between hosts (H2); link costs are always measured on the target pairing. Copies and kernels on one stream are serialised; every trace is checked for copy-kernel concurrency and unified-memory faults. Contention is measured, never fitted.

Baselines from the same components.

- **Sum:** $\text{sum}(h) + \text{sum}(l) + \text{sum}(d)$, no overlap.
- **KernelSight-style step form:** $\max(\text{sum } h, \text{sum } l + \text{sum } d)$ per layer, plus every synchronisation cost as a serial tail. It differs from the recursion only through operation order, synchronisation positions and the queue term, so the gap between the two isolates G2.
- **Learned (CTFusion-style LightGBM):** task features (context length, density, batch, operation and synchronisation counts), hardware microbenchmark features (host, link, device costs; pinned and pageable bandwidth) and bottleneck ratios (arithmetic intensity per operation class, host-to-device time ratio, transfer demand relative to link bandwidth, synchronisations per operation); trained leave-one-pairing-out; features and hyperparameters fixed at lock. Tests whether the structure adds anything over regression (G1).
- **HDBI** reported as a diagnostic only.

Batch. Batch 1 and 4 at 16384 on the L4 pairings; batch 4 at 32768 where memory allows on the A100 and H100. **Crossover.** Sparse is profitable where $D_N(\text{sparse}) < D_N(\text{dense})$; solving for context length gives each estimator's crossover per pairing. **Queue sensitivity.** Every committed prediction is recomputed with Q at 0.25x, 0.5x, 2x and 4x the measured value, and with no queue term.

5.5 Arms

Arm	What it does	Role
Dense	Fastest correct kernel per card and context length (FA3 on the H100)	Reference
Oracle	Pools full dense-softmax attention mass to blocks, from a separate dense pass (one ranking per KV head, meaned over its query heads)	Attention-mass reference; highest cost. Not a guaranteed quality ceiling
Window	Sink plus local blocks	Zero-cost floor
Mean-pool	Pooled query and key blocks (MInference-style); inline device builder	Cheap deployable baseline
Vertical-slash style	Vertical and slash scoring reduced to blocks	Different estimator structure
Calibrated XAttention	Antidiagonal scoring with per-(layer, head) thresholds from the authors' profiler, run as shipped including its GPU-CPU synchronisations	Data-dependent density and synchronisations

Arm	What it does	Role
DuoAttention-style (conditional)	Retrieval heads dense, streaming heads sink plus local, from released head patterns; only if patterns exist for a study model and the licence allows	Head-differentiated recall profile for H4
Precomputed-mask (ablation)	Each estimator's masks computed offline and preloaded; then the same kernel	Separates estimator compute and synchronisation from kernel and launch cost

All arms use one per-head selector and one block-sparse kernel. A bitwise gate requires the selector to reproduce the head-uniform rule exactly when heads share scores. Timed block size is 128.

5.6 Confounders and controls

Confounder	Control
CPU microarchitecture, caches, memory bandwidth	Inside c_host, measured on the target host at target sizes; never synthesised
PCIe generation, copy and sync latency	c_link measured per pairing; generation and width recorded
Cloud CPU frequency (governor and true clock often hidden)	Host-speed probe distributions per session; a session is usable only if the probe's coefficient of variation is under a bound; a component transfers only if source and target probe medians differ by less than a bound. Both bounds fixed at lock from P1 repeat sessions. Failing sessions are rerun
vCPU topology, SMT, noisy neighbours	Pinning; topology and tenancy recorded; probe detects drift; P1 repeat sessions give the between-session floor
Host-memory contention between mask building and DMA	Link bandwidth also measured under mask-sized CPU load; copies in flight during host-side estimator work are costed at the loaded bandwidth
Driver and software differences	One container image; driver recorded; dense control every session
Measurement asymmetry between arms	One device-time method (isolated replay) for all arms; graph replay only as a cross-check
Prediction made trivial	No prediction input comes from the composed workload on the predicted pairing
Profiler overhead	Ground truth from untraced runs
Predictions fitted after the fact (K9)	Predictions hashed and synced before the workload runs
NUMA placement	Threads pinned to the GPU's local node; microbenchmarks from that node
Pinned vs pageable memory	Both measured; each traced copy classified and costed
Launch-queue backpressure	Q measured per pairing; host costs measured in bursts
Unified memory	Not used; any fault flags the row
Replay mismatch	Replayed kernel names and durations compared with the P1 trace per operation family before H1 is read; mismatched variants flagged
Null-kernel floor plausibility	P4's floor reported with its exact definition and compared only with a published figure measured the same way (TaxBreak about 4.7 us on Xeon 8480C with H100; SKIP 2.37 us on Xeon 8468V with H100 PCIe)

Study controls K1 to K9. K1 one selector and kernel for every arm (built); K2 fastest correct dense baseline per card (drafted); K3 disjoint calibration splits with a pre-run gate (built); K4 three models, Qwen2.5-1.5B, Qwen2.5-7B, Llama-3.1-8B held out of all fitting (planned); K5 recall at block sizes 16 to 128 at matched density (control only; AB-Sparse already reports the decode-side version); K6 pre-registration, checks that fail on purpose, provenance on every row (in place); K7 cells count only if dense accuracy clears a floor (not built); K8 code path recorded per row, fallbacks never on the same curve (built); K9 predictions hashed before the workload (not built).

5.7 Quality measures

- **Accuracy test (proposed, confirm at lock):** exact paired non-inferiority (union-bound Clopper-Pearson), margin 10 percentage points, one-sided alpha 0.025, as used in the pilots. The dense arm must reproduce exactly in every run.
- **Recall:** both references (Section 5.1), per layer, on the very masks that produced each accuracy row; also by head type (retrieval-like vs local, criterion fixed at lock) and by depth third.
- **Intrinsic recall:** budgets 0.50, 0.25, 0.10, 0.05; null scorer as a floor; budget selection on 32 examples per task.
- **H4 across block sizes:** on the primary cell (qa_1/16384, n = 100), block sizes 32, 64 and 128 through an untimed reference masked-attention path; block 16 only if that path supports it.
- **Where measured:** once, on the primary GPU, with the dense control repeated on a second GPU.
- **Dense floor (K7)** and **critical density** (lowest non-inferior oracle density; descriptive only).

5.8 Extension: chunked prefill

On P5, with inline mean-pool and calibrated XAttention, prefill also runs in chunks of 2048 and 4096 tokens at 16384 and 32768. Each chunk reruns the estimator over the accumulated keys, which the recursion represents as repeated operations; the predicted re-selection overhead is committed first. First check whether the block-sparse kernel supports query chunks shorter than the key length; if not, drop the extension and record it. The kernel's time at each query-to-key ratio is recorded separately from the per-chunk estimator operations, and mask density separately from executed density. External reference only: CompactAttention reports XAttention at 0.68x to 0.84x of dense speed end to end under chunked prefill on two GPUs.

6. Operating rules (both Claudes, every session)

Claims and evidence

1. Never claim "no accuracy loss" or "zero accuracy cost". Report non-inferiority at a stated margin.
2. Keep oracle, deployable-estimator, CPU-builder and GPU-builder results distinct; never put them on one curve.
3. Every measurement is commit-stamped with `git_dirty = False`; results from different mask-rule eras or code paths are never pooled.
4. Never delete or rewrite historical audit findings. Withdrawn or corrected items are annotated in place.
5. Cite only papers read in full; anything else is marked "from abstract". If a paper is needed and not available, ask the researcher for the PDF instead of characterising it from search summaries.
6. Predictions are written and hashed before the data they predict.

Code

7. Run the focused tests for anything changed before committing; keep the full suite green.
8. Do not commit unrelated changes. Third-party code is installed and called, never copied.
9. No model identifiers or assistant attribution in research artifacts beyond the commit trailers the tooling adds.

Cloud sessions (GCP now, AWS later)

10. Before any GPU session, state the exact command, estimated cost, cost cap, hard-delete limit and artifact path, and wait for approval. The researcher runs all cloud commands from their Mac.
11. Launch sequence: cleanup check, cost cap, hard-delete limit, stale-results quarantine, deploy the clean commit, preflight, run, teardown through the teardown script (which writes `session_cost.txt`). Update `docs/spend_ledger.md` from that file.
12. L4 zone `asia-northeast1-a` (fallback zones as in the launch scripts). Never an Australia region.

13. AWS sessions only after the AWS tooling meets the same guarantees and passes tests against a fake CLI (task T5.1).

Writing for the professor: plain language, short sentences, no em dashes.

7. Decision gates

Gate	Condition	If it fails
Gate A: P1	The recursion beats the sum and step-form baselines on P1 (T3.3)	Stop; revise the model before any new pairing
Gate B: lock	All [lock] values fixed, two adversarial reads done, lock commit tagged (T4.x)	No confirmatory pairing runs
Gate C: AWS	AWS tooling passes fake-CLI tests; quotas granted; each pairing costed	No AWS launch
Gate D: H0	Separability holds	H2 not tested; scope narrows to per-pairing composition
Budget	Each session within its cap	Cut in an order fixed at lock (proposed: P7 first, then the chunked extension); the cut order never removes ground-truth timing, H1 or H4

8. Task list

Owner: **Code** = Claude Code, **Chat** = Claude chat with the researcher, **R** = researcher. IDs are permanent; do not renumber. Add new tasks at the end of a phase.

Phase 0: Repository transition

ID	Task	Owner	Depends	Done when	Status
T0.1	Commit this plan, its PDF, CLAUDE.md and the build tool	Code		On main	In progress (on branch; merge pending)
T0.2	Brief Claude chat: upload the PDF to the project, remove superseded plan PDFs, paste the handover note	R	T0.1	Chat answers a test question from the plan correctly	Not started
T0.3	Tag the pre-pivot state (pre-pivot-2026-10-04) so every banked result stays reproducible	Code (R approves push)		Tag on origin	Not started
T0.4	Review docs/code_inventory.md: confirm keep / adapt / archive for every module and script	R + Chat decide; Code verifies by reading the code	T0.1	Every row has a confirmed decision	Not started
T0.5	Move archived code and its tests to legacy/ (not deleted), fix imports, keep the suite green	Code	T0.3, T0.4	Suite green; no live module imports legacy/	Not started
T0.6	Move superseded docs to docs/archive/ unchanged; update test references to their paths	Code	T0.3	Suite green	Not started
T0.7	Rewrite README.md for the new question; remove claims that break rule 1	Code	T0.1	README matches Sections 1 and 2	Not started
T0.8	Point .github/copilot-instructions.md at this plan	Code	T0.1	Done	Not started
T0.9	Carried-over housekeeping: ledger - updates branch at b00019f, spend ledger brought up to date, audit addenda	Code		Confirmed with R and done	Decision (scope to confirm)

ID	Task	Owner	Depends	Done when	Status
T0.10	Archive the last Estimator Frontier plan (with referee responses) and the professor brief in docs/archive/ and docs/	Code	T0.1	Files committed	Done

Phase 1: Quality evidence (G3) on existing infrastructure

ID	Task	Owner	Depends	Done when	Status
T1.1	Run calibrated XAttention sessions A and B as pre-registered (docs/t4_xattention_calibrated.md)	Code prepares; R runs		Rows banked, analysis written	Built, not run
T1.2	Recall with both references, per layer, by head type and depth third, on accuracy rows	Code		Tested on CPU fixtures; GPU check at 16384	Partly built (intrinsic recall pass exists)
T1.3	Fix the head-type criterion (retrieval-like vs local)	Chat decides; Code implements		Criterion in code and here	Decision
T1.4	Wire the reference masked path (backends/block_masked.py) into H4 runs at block sizes 32 and 64	Code		Accuracy rows at 32, 64, 128 on a dry run	Partly built
T1.5	H4 dry analysis on banked pilot rows plus recall (exploratory, not confirmatory)	Code	T1.2	Report written, marked exploratory	Not started
T1.6	Cost the real-text haystack swap on the primary model; decide	Code costs; R decides		Decision recorded	Not started
T1.7	Build the window and vertical-slash arms; check DuoAttention patterns and licence	Code		Arms pass the selector gate	Not started

Phase 2: Component measurement and the model

ID	Task	Owner	Depends	Done when	Status
T2.1	Host-speed probe (single-thread loop + null-kernel burst), adapting scripts/host_cpu_probe.py	Code		Runs on CPU; GPU path dry-run tested	Partly built
T2.2	Null-kernel host-component profiler (TaxBreak-style)	Code		Unit-tested; L4 smoke run	Not started
T2.3	Operation sequence and synchronisation map from a P1 trace; checker against any pairing's kernel list	Code		Tested on a recorded trace	Not started
T2.4	Isolated per-operation device replay with recorded masks; graph-replay cross-check	Code	T2.3	Replay vs trace check passes on P1	Not started
T2.5	Link microbenchmarks: pinned and pageable bandwidth, 1-byte copy, sync round trip, PCIe, loaded bandwidth, NUMA node	Code		All fields in one provenance-stamped row	Partly built (launch, sync, 1-byte copy; measure_mask_h2d_tax.py)
T2.6	Launch-queue depth probe	Code		Q measured on the L4	Not started
T2.7	Trace checks: copy-kernel concurrency, unified-memory faults	Code		Flags tested on synthetic traces	Not started
T2.8	The recursion, with hand-computed unit tests	Code		Tests pass	Not started
T2.9	Sum and KernelSight-style baselines; HDBI diagnostic	Code	T2.8	Tests pass	Not started

ID	Task	Owner	Depends	Done when	Status
T2.10	Learned LightGBM baseline, leave-one-pairing-out	Code	T2.8	Features frozen in code	Not started
T2.11	Prediction commit: hash, sync, analysis refuses late predictions (K9)	Code	T2.8	Refusal tested	Not started
T2.12	Precomputed-mask ablation arm	Code		Passes selector gate	Not started
T2.13	Untraced ground-truth harness: arms interleaved, dense control, traced-vs-untraced overhead (adapt run_vectorised_endtoend.py / phase_timing.py)	Code		L4 dry run	Partly built
T2.14	One container image for GCP and AWS	Code		Boots on both	Not started
T2.15	Queue-sensitivity and residual analysis code	Code	T2.8	Tested	Not started

Phase 3: P1 gate

ID	Task	Owner	Depends	Done when	Status
T3.1	P1 session: components, committed prediction, ground truth	Code prepares; R runs	Phase 2	Rows banked	Not started
T3.2	P1 repeat sessions for between-session floors; set probe bounds	Code + R	T3.1	Bounds proposed for lock	Not started
T3.3	Gate A decision	Chat + R	T3.1	Recorded in Appendix A	Not started

Phase 4: Pre-registration lock

ID	Task	Owner	Depends	Done when	Status
T4.1	Fix every [lock] value: H0 to H3 tolerances, H4 equivalence bound, probe bounds, resolution floor, budget cut order	Chat + R	T3.2	Values written here	Not started
T4.2	Confirm the accuracy test and the dense floor (K7)	Chat + R		Written here	Decision
T4.3	Two independent adversarial reads of plan and scoring code	R arranges	T4.1	Findings resolved	Not started
T4.4	Tagged lock commit	Code	T4.3	Tag on origin	Not started

Phase 5: AWS tooling

ID	Task	Owner	Depends	Done when	Status
T5.1	AWS launch, cleanup, cost cap, hard-delete limit, teardown with the GCP guarantees; tests against a fake CLI	Code		Tests pass	Not started
T5.2	Service quotas for G6, G6e, p5.4xlarge, G5; regional availability	R		Quotas granted	Not started
T5.3	Cost every pairing session	Code	T5.1	Table in docs/spend_ledger.md	Not started

Phase 6: Pairings (each with its prediction committed first)

ID	Task	Owner	Depends	Status
T6.1	P2 (Cascade Lake + A100): H0, H1, H2	Code prepares; R runs	Gate B	Not started
T6.2	P3 (AMD + L4): H0, H1, H2	Code; R	Gates B, C	Not started
T6.3	P5 (AMD + L40S): H0, H1, H2, H5; chunked extension	Code; R	Gates B, C	Not started

ID	Task	Owner	Depends	Status
T6.4	P4 (Sapphire Rapids + H100): H1, H3	Code; R	Gate B	Not started
T6.5	P6 (AMD + H100): H0, H1, full transfer, H5	Code; R	T6.2 to T6.4	Not started
T6.6	P7 (AMD 2nd gen + A10G), optional	Code; R	Budget	Not started

Phase 7: Analysis and writing

ID	Task	Owner	Depends	Status
T7.1	H0 to H5 analyses per pairing; residuals by arm and synchronisation count	Code	Phase 6	Not started
T7.2	Read in full the related work still known only from abstracts (Token Sparse Attention, nn-Meter, HELP, Twilight, FlexPrefill, others in the brief)	Chat (R supplies PDFs)		Not started
T7.3	IEEE Xplore and Scopus search; related-work section	Chat + R	T7.2	Not started
T7.4	Venue choice (Section 11)	R with professor		Not started
T7.5	Paper draft; professor review	R + Chat	T7.1	Not started

9. Known risks

- **Small panel.** Seven pairings; claims are limited to the measured host platforms and GPUs.
- **Separability may fail** (Gate D).
- **Component profiling may misstate composed behaviour.** Null kernels can understate host cost; isolated replay removes cache and queue interactions; TaxBreak notes replay is imperfect for synchronisation-heavy kernels, which describes our sparse arms. H1's error on real pairings measures how much this matters.
- **Profitability may be empty for deployable estimators.** In the pilots none was non-inferior at any tested density. The primary claim is time prediction at a given density.
- **The learned baseline may win.** It has at most six training pairings per held-out test. Both outcomes are reported, on held-out pairings only.
- **G3 may be partly pre-empted.** Oracle-guided sparse prefill plans to measure recall of oracle support by its learned indexer. H4 is the cross-estimator test; that is how it is positioned if their follow-up appears first.
- **H4 may fail.** That is a result.
- **Cloud hosts are not controllable.** A frequency difference between sessions is the first candidate explanation for any H0 failure.
- **Two providers** add operational risk (Gate C).
- **Sparse speedups are small on the primary model.** The claims concern prediction.

10. Out of scope, and rejected alternatives

Out of scope: models needing more than one GPU; decode sparsity; natively trainable sparse attention; continuous batching and batch above 4; chunked prefill beyond Section 5.8; synthetic host speeds as evidence (delay injection only as an instrument check); a new estimator as a headline; timed kernels below block 128; cross-attention; linear attention; contexts above 65K; model families beyond Qwen2.5 and Llama-3.1.

Rejected	Reason
A head-adaptive or entropy-driven estimator as the contribution	Already explored (FlexPrefill, Twilight, AB-Sparse)
Recall by block size per head as a contribution	AB-Sparse reports it; kept as control K5
Benchmarking estimator cost against quality	Method papers and CompactAttention already report it
Oracle masks as a quality ceiling	An attention-mass oracle is a reference, not a task optimum (oracle-guided sparse prefill; The Sparse Frontier)
An analytical sparse-kernel model carried across GPU generations	KernelSight-LM finds attention-kernel efficiency does not transfer; device costs are measured
A fitted contention penalty	Would be tuned on the outcomes it predicts
A serving simulator as the baseline	Both simulators use per-device profiles and model no host work inside a step; the learned predictor and step form are fairer comparators
Q sensitivity over {4, 8, 16}	Far below the measured queue scale; sensitivity is relative to measured Q
PLA's depth-wise concentration as evidence against H4	H4 already matches recall per layer; depth is reported as a stratum

11. Open questions (for the professor or a reviewer)

1. Does the crossed seven-pairing design answer the validation objection?
2. Is the definition of an unmeasured pairing acceptable, and is P6's full transfer convincing?
3. Are sum, step form and LightGBM the right comparators; is leave-one-pairing-out on seven pairings fair to the learned one?
4. Is out-of-sample time prediction enough for an IEEE Transactions journal, or is a design contribution also needed (for example a selector driven by the model's predictions)?
5. If the model fails out of sample, is a well-characterised negative result publishable at the target venue?
6. Which venue: IEEE TC, TPDS, TCAD or ACM TACO; or ISPASS or MLSys first?

12. Repository map

Path	What it is
docs/RESEARCH_PLAN.md	This plan (source); docs/research_plan.pdf is generated from it
docs/research_brief.pdf	Short brief for the professor, with the related-work table
docs/code_inventory.md	Keep / adapt / archive review of the existing code (task T0.4)
docs/spend_ledger.md	Cloud spend, from teardown-written session_cost.txt files
docs/limitations.md, docs/claims.md, docs/withdrawn_figures.md, docs/silent_failure_patterns.md	Audit history of the banked evidence; annotate, never rewrite
docs/archive/	Superseded plans and their referee responses
attnbench/	Harness: provenance, timing, masks, backends, accuracy pipeline, analysis
scripts/	Session runners and GCP tooling
tools/build_plan_pdf.py	Renders this file to PDF

Appendix A. Decision record (dated, one line each)

- **4 Oct 2026.** Study direction set to predicting when sparse prefill pays across real host-GPU pairings; G2 and G3 lead, G1 is the validation question. Validation only on physically distinct pairings; synthetic host delay is never evidence.
- **4 Oct 2026.** Link cost is its own term; only device costs transfer between hosts.
- **4 Oct 2026.** Cloud CPU frequency is controlled by a measured host-speed probe with bounds fixed from P1 repeat sessions, not by frequency bands.
- **4 Oct 2026.** Baselines: sum, KernelSight-style step form, CTFusion-style LightGBM (leave-one-pairing-out). Serving simulators declined as baselines.
- **4 Oct 2026.** Budget extended by the researcher to cover the AWS pairings (P3, P5, P6, optional P7); each session still capped.
- **4 Oct 2026.** The oracle arm is an attention-mass reference, not a quality ceiling. Recall's primary reference is the arm's own forward pass; the separate dense pass is secondary.
- **4 Oct 2026.** H4 reported by depth third and by head type; recall matched per layer.
- **4 Oct 2026.** Launch-queue sensitivity is relative to measured Q (0.25x to 4x, and none).
- **4 Oct 2026.** SparkV withdrawn by its authors; not cited.
- **4 Oct 2026.** This document replaces the Estimator Frontier plan as the single source of truth.

Appendix B. Glossary

Arm: one way of producing masks (or dense attention), timed and scored identically. **Estimator:** the part of an arm that decides which blocks to keep. **Density:** fraction of blocks kept; *executed* density can differ from *mask* density after unions. **Crossover:** the context length above which sparse beats dense on a pairing. **Pairing:** one host CPU platform with one GPU model on one provider. **Mask-rule era:** a period during which the mask rule was unchanged; results from different eras are not pooled.